L06: Embeddings & RL

Deep Dive: Theory, Implementation, and Applications

Methods and Algorithms

MSc Data Science

Spring 2026

Outline

Pouring Data into Linear Algebra

XKCD #1838 "Machine Learning" by Randall Munroe (CC BY-NC 2.5)

Learning Objectives

After this lecture, you will be able to:

1. **Derive** the Skip-Gram objective and negative sampling approximation
2. **Evaluate** static vs contextual embeddings (Word2Vec, GloVe, FinBERT)
3. **Analyze** Q-learning convergence and the exploration-exploitation trade-off
4. **Critique** RL trading strategies (transaction costs, non-stationarity, overfitting)

Finance Applications: Sentiment analysis with embeddings, algorithmic trading with RL

Bloom's Levels 4–5: Analyze, Evaluate, Derive, Critique

Why Can't One-Hot Vectors Capture Meaning?

The Problem with One-Hot Encoding

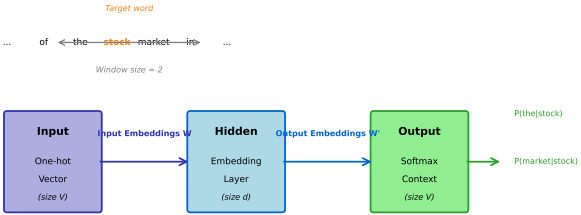
- Vocabulary of 10,000 words $\rightarrow$ 10,000-dim sparse vectors
- No semantic similarity: "king" and "queen" equally distant from "car"
- Curse of dimensionality

Solution: Dense Embeddings

- Map words to dense vectors (50-300 dimensions)
- Similar words $\rightarrow$ similar vectors (via cosine similarity—defined later)
- Learn from context (distributional hypothesis)

"You shall know a word by the company it keeps" – Firth, 1957

Skip-gram Architecture



Skip-gram Architecture: Predict Context from Target

Two embedding matrices: input W (word vectors) and output W' (context vectors)

How Does Skip-Gram Learn Word Meaning?

Objective: Predict context words given target word

$$P(w_{context}|w_{target}) = \frac{\exp(v_{context}^T v_{target})}{\sum_{w \in V} \exp(v_w^T v_{target})}$$

Training:

- Slide window over text corpus
- For each word, predict surrounding words
- Update embeddings via gradient descent

Skip-gram works well for rare words; CBOW better for frequent words

Why Is Full Softmax Too Expensive?

Problem: The softmax denominator sums over **entire vocabulary**:

$$\sum_{w \in V} \exp(v_w^T v_{target}) \quad \text{— } O(|V|) \text{ per update!}$$

For $|V| = 100{,}000$ words, this is computationally intractable.

Solution: Negative Sampling (Mikolov et al., 2013b)
Replace full softmax with binary classification:

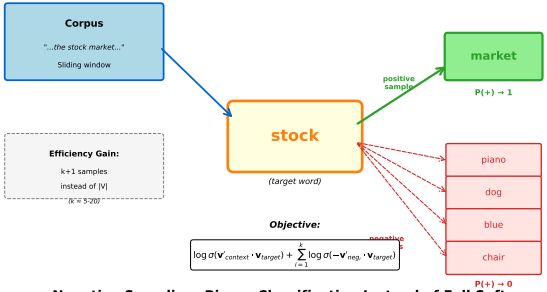
$$\log \sigma({v'_{w_O}}^T v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n} \left[\log \sigma(-{v'_{w_i}}^T v_{w_I}) \right]$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function.

- Positive pair: (target, true context) $\rightarrow$ predict 1
- k negative pairs: (target, random word) $\rightarrow$ predict 0
- Reduces $O(|V|)$ to $O(k)$ where $k = 5$–20 for small datasets; 2–5 for large corpora

Negative sampling: the key innovation that made Word2Vec practical

Negative Sampling Illustrated



$$\log \sigma(\mathbf{v}'_{context} \cdot \mathbf{v}_{target}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}'_{neg_i} \cdot \mathbf{v}_{target})$$

Negative Sampling: Binary Classification Instead of Full Softmax

Binary classification: distinguish true context words from random "noise" words

How Does Negative Sampling Work in Practice?

Require: corpus, embedding dim d, negatives k, window size, epochs

```
Initialize W, W' ∈ ℝ^{|V|×d} randomly
for each epoch do
    for each word w_t in corpus do
        for each context word w_c within window do
            Positive: update (w_t, w_c) to increase σ(v_{w_t}^⊤ v'_{w_c})
            for i = 1, ..., k do
                Sample w_n ~ P_n(w) ∝ f(w)^{3/4} {f(w): word freq.}
                Negative: update (w_t, w_n) to decrease σ(v_{w_t}^⊤ v'_{w_n})
            end for
        end for
    end for
end for
return W (word embeddings)
```

Key: Negative sampling (3-5 negatives per positive) replaces expensive softmax over entire vocabulary.

Mikolov et al. (2013). Distributed representations of words and phrases. NeurIPS, 3111–3119.

Can Vectors Solve Analogies?

Famous Example:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$$

Finance Examples:

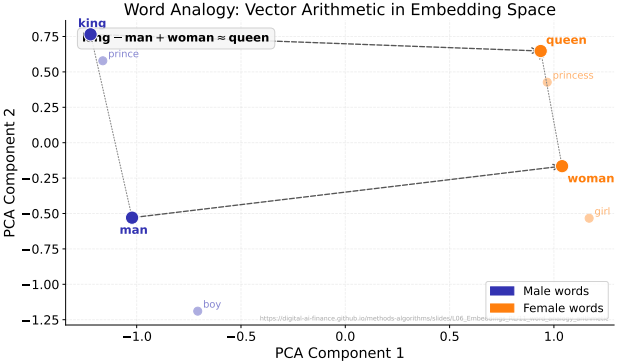
- $\vec{stock} - \vec{equity} + \vec{debt} \approx \vec{bond}$
- $\vec{CEO} - \vec{company} + \vec{country} \approx \vec{president}$

How it works:

- Vector arithmetic in embedding space
- Relationships encoded as directions

Embeddings capture relational structure, not just similarity

Word Analogy: Vector Arithmetic



Vector arithmetic in embedding space: relationships encoded as directions (PCA projection of 50-dim embeddings)

When Do Word Analogies Fail?

Known Issues:

- Success rates typically 40–70%, not near 100% (Levy & Goldberg, 2014)
- Evaluation methodology inflates accuracy (nearest-neighbor dominance)
- Finance-domain analogies ($\vec{stock} - \vec{equity} + \vec{debt} \approx \vec{bond}$) not empirically validated

Bias in Embeddings:

- Embeddings encode societal biases from training data (Bolukbasi et al., 2016)
- Example: man:programmer :: woman:homemaker
- **Finance concern**: Biased embeddings in credit scoring or hiring tools

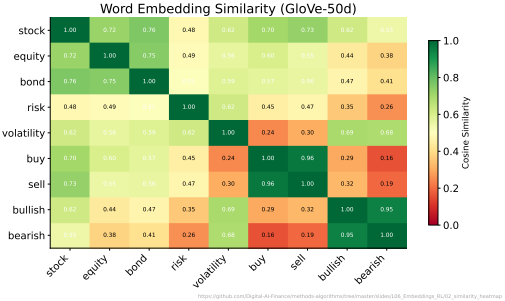
Critical thinking: embeddings capture statistical patterns, including harmful ones

How Do We Measure Closeness in Vector Space?

Cosine Similarity:

$$\text{sim}(u, v) = \frac{u \cdot v}{||u|| \cdot ||v||} = \cos(\theta)$$

- Range: $[-1, 1]$; 1=same direction, 0=orthogonal, -1=opposite



Cosine similarity ignores magnitude, focuses on direction

Should You Train Your Own Embeddings?

Popular Options:

- **Word2Vec**: Google, 300-dim, 3M words
- **GloVe**: Stanford, trained on Wikipedia + Common Crawl
- **FastText**: Facebook, handles subwords (OOV robust)

Domain-Specific:

- **FinBERT**: BERT further pre-trained on financial corpora (Araci, 2019)
- BioBERT: Biomedical domain

Fine-tuning pre-trained embeddings usually outperforms training from scratch

Does Context Change a Word's Meaning?

Static Embeddings (Word2Vec, GloVe, FastText):

- ONE fixed vector per word, regardless of context
- "bank" in "river bank" = "bank" in "bank account"
- Fast, simple, good baseline

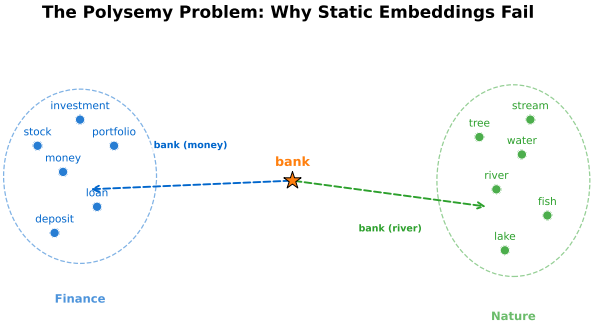
Contextual Embeddings (BERT, GPT, FinBERT):

- DIFFERENT vector per occurrence based on surrounding context
- "bank" gets different representations in different sentences
- State-of-the-art for most NLP tasks

Key Insight: Contextual models solve polysemy (multiple word senses)

Static: one meaning per word. Contextual: meaning depends on context.

Can You See Why Static Embeddings Fail?



Static: 1 vector for "bank". Contextual (BERT): different vectors based on meaning.

- "Bank" sits between two clusters — the model can't decide which meaning
- Contextual models solve this by producing different vectors per context
- The dashed arrows show where BERT would place each sense of "bank"

This visualization uses synthetic coordinates but reflects real embedding space behavior

How Do Banks Use Word Embeddings?

Applications:

- **Sentiment Analysis**: News → embedding → positive/negative
- **Document Similarity**: Find similar SEC filings
- **Named Entity Recognition**: Extract company names
- **Event Detection**: Identify earnings announcements

Sentence Embeddings:

- Average word vectors (simple but loses word order: "bank robber" = "robber bank")
- Doc2Vec (paragraph vectors)
- Sentence-BERT (state-of-the-art)

Aggregate word embeddings to represent documents

How Would You Classify This Headline?

Task: Classify "Fed signals rate hike" as positive or negative

Step 1: Average word embeddings (simplified 3-dim vectors):

$$\vec{v}_{\text{sentence}} = \frac{1}{4}(\vec{v}_{\text{Fed}} + \vec{v}_{\text{signals}} + \vec{v}_{\text{rate}} + \vec{v}_{\text{hike}}) = [0.12, -0.31, 0.45]$$

Step 2: Compare to sentiment anchors via cosine similarity:

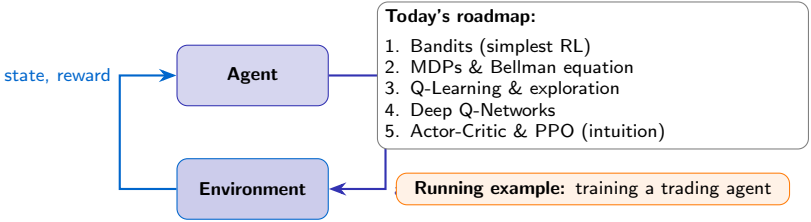
- $\text{sim}(\vec{v}_{\text{sentence}}, \vec{v}_{\text{positive}}) = 0.23$
- $\text{sim}(\vec{v}_{\text{sentence}}, \vec{v}_{\text{negative}}) = 0.61$

Step 3: Classify: **Negative sentiment** (rate hikes $\rightarrow$ tighter policy)

Real-world: Use FinBERT for production sentiment (up to 87% accuracy on financial text; Araci, 2019)

Simplified example — real embeddings are 300-768 dimensions with learned sentiment structure

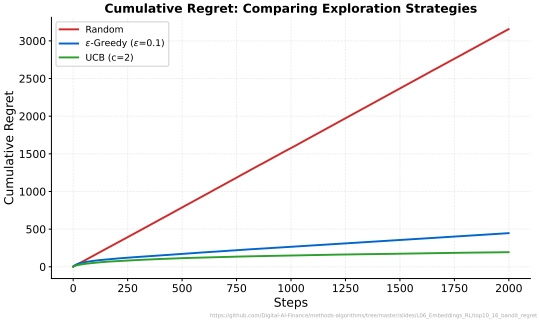
What Does the RL Landscape Look Like?



This roadmap covers the RL section. We progress from the simplest case (bandits) to modern methods (PPO).

What If There Is Only One State?

The Multi-Armed Bandit Problem



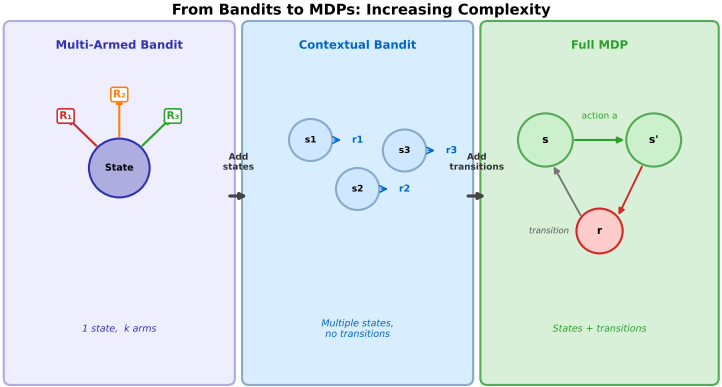
- k actions ("arms"), each with unknown reward distribution
- Goal: maximize total reward over T rounds
- **Regret:** $\text{Regret}_T = T \cdot \mu^* - \sum_{t=1}^{T} r_t$
- Finance: "Which of k trading strategies to allocate capital to?"

Key Insight

Bandits are RL with **no state transitions** — the simplest case.

Bandits are the gateway to full RL. Next: add state transitions to get MDPs.

From Bandits to MDPs: Adding Complexity Step by Step



- **Bandit:** One state, k actions — just explore vs. exploit
- **Contextual bandit:** State matters, but actions don't change the state
- **Full MDP:** Actions cause state transitions — now the future depends on your choices

Each level adds complexity. The MDP formalism (next slide) captures the full problem.

What Are the Building Blocks of RL?

Key Components:

- **Agent**: Learner/decision-maker
- **Environment**: What agent interacts with
- **State** s: Current situation
- **Action** a: What agent can do
- **Reward** r: Feedback signal

The RL Loop:

Agent observes State $\rightarrow$ Agent selects Action $\rightarrow$ Environment transitions $\rightarrow$ Environment emits Reward $\rightarrow$ Agent updates $\rightarrow$ (repeat)

RL: Learning from interaction, not from labeled examples

What Makes a Problem Markovian?

MDP Tuple: (S, A, P, R, γ)

- S: Set of states
- A: Set of actions
- $P(s'|s, a)$: Transition probability
- $R(s, a, s')$: Reward function
- $\gamma \in [0, 1)$: Discount factor (or $\gamma \in [0, 1]$ for episodic tasks)

Markov Property:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, ...) = P(s_{t+1}|s_t, a_t)$$

Future depends only on current state, not history

How Do We Measure a Strategy's Worth?

Policy: $\pi(a|s) = P(A_t = a|S_t = s)$

- Maps states to action probabilities
- Goal: Find optimal policy π^*

Value Function (R_{t+1}: reward at step $t+1$):

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s \right]$$

Q-Function (Action-Value):

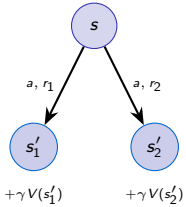
$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s, A_0 = a \right]$$

Q-function: expected return starting from state s, taking action a

Why Is the Bellman Equation Recursive?

Motivate: Evaluating a policy requires summing infinite future rewards — impossible directly.

Visualize: One-step lookahead:



Formalize:

$$Q^*(s,a) = \mathbb{E}\left[R + \gamma \max_{a'} Q^*(s',a')\right]$$

Trading example ($\gamma = 0.9$):

- Cash $\xrightarrow{\text{buy}}$ Holding: $r = -0.01$ (commission)
- Holding $\xrightarrow{\text{sell}}$ Cash: $r = +0.10$ (profit)
- $Q(\text{Cash}, \text{buy}) = -0.01 + 0.9 \times 0.10 = 0.08$

Bellman equation: value = immediate reward + discounted future. Foundation of all value-based RL.

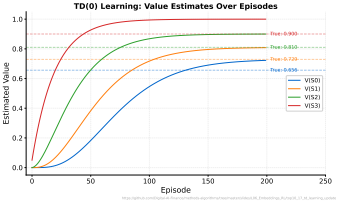
Can We Learn Without Waiting for the End?

TD(0) Update Rule (learning rate $\alpha \in (0, 1]$) — learn from each transition:

$$V(s) \leftarrow V(s) + \alpha \left[r + \gamma V(s') - V(s) \right]$$

TD Error: $\delta_t = r + \gamma V(s') - V(s)$ (surprise signal)

- **vs Monte Carlo**: MC waits for episode end; TD updates every step
- **vs Dynamic Programming**: DP requires model $P(s'|s, a)$; TD is model-free
- **Q-learning**: TD applied to Q-function with max over actions



TD learning: the theoretical foundation connecting DP, MC, and Q-learning (Sutton, 1988)

How Does Q-Learning Find the Best Action?

Update Rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Algorithm:

1. Initialize $Q(s,a)$ arbitrarily
2. For each episode:
   - Observe state s
   - Choose action a (ϵ-greedy: random with prob. ϵ, best otherwise)
   - Execute a, observe r, s'
   - Update $Q(s,a)$

Q-learning is off-policy: converges to Q^* given Robbins-Monro conditions ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$) and sufficient exploration (Watkins & Dayan, 1992)

What Happens in One Q-Learning Update?

Trading scenario: State s_1 = [RSI=25, position=none]

Current Q-values: $Q(s_1, \text{buy}) = 3.2$, $Q(s_1, \text{hold}) = 1.0$

Agent takes action **buy**, observes:

- Reward $r = -0.5$ (transaction cost)
- New state s_2 = [RSI=35, position=long]
- Best future: $\max_{a'} Q(s_2, a') = 4.0$

Update ($\alpha = 0.1$, $\gamma = 0.9$):

$$\underbrace{r + \gamma \max_{a'} Q(s_2, a')}_{\text{TD target}} - \underbrace{Q(s_1, \text{buy})}_{\text{current}} = -0.5 + 0.9 \times 4.0 - 3.2 = -0.1$$

$$Q(s_1, \text{buy}) \leftarrow 3.2 + 0.1 \times (-0.1) = \mathbf{3.19}$$

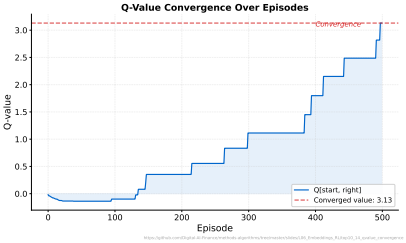
Each update moves Q toward the "better" estimate: immediate reward + discounted future

Q-Learning Algorithm: Pseudocode

Require: environment, α, γ, ϵ, episodes

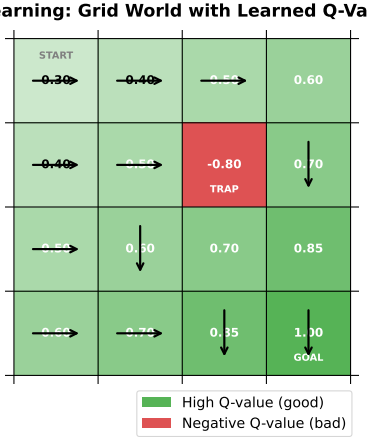
Initialize $Q(s,a) \leftarrow 0$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}$
for episode $= 1, \ldots,$ episodes **do**
  $s \leftarrow$ initial state
  while s is not terminal **do**
    $a \leftarrow \begin{cases} \text{random } a \in \mathcal{A} & \text{with prob. } \epsilon \\ \arg\max_{a'} Q(s,a') & \text{otherwise} \end{cases} \quad \{\epsilon\text{-greedy}\}$
    Take action a, observe reward r and next state s'
    $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$
    $s \leftarrow s'$
  end while
end for
return Q

Key: The $\max_{a'}$ makes Q-learning **off-policy** — it learns the optimal policy regardless of the exploration strategy used.

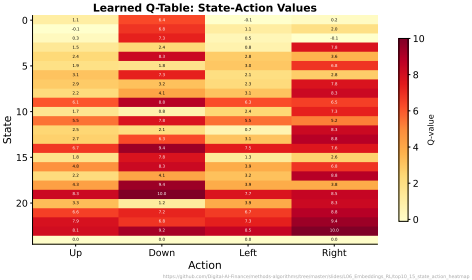


Watkins & Dayan (1992). Q-learning. Machine Learning, 8(3-4), 279–292.

Q-Values Visualization



https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/04_q_learning_grid



https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/top10_15_state_action_heatmap

Left: arrows show policy; colors show Q-values (green=high, red=negative). Right: state-action Q-value heatmap.

Explore or Exploit: How Do You Choose?

The Dilemma:

- **Exploit**: Choose best known action (greedy)
- **Explore**: Try new actions (discover better options)

ϵ-Greedy Strategy:

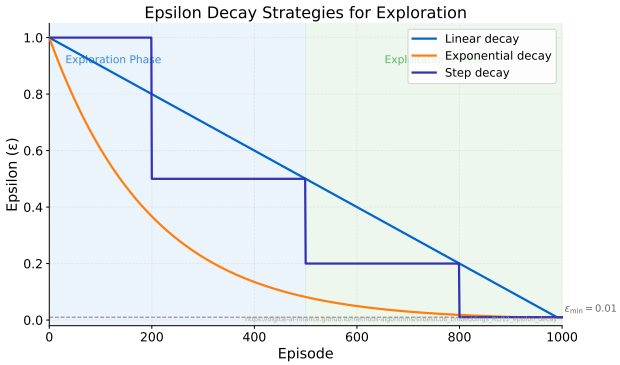
$$a = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases}$$

Decay Schedule:

- Start with high ϵ (explore more)
- Decay ϵ over time (exploit more)

Balance: too much exploration wastes time; too little misses optima

Epsilon Decay Schedule



Decay Strategies:

- **Linear:** $\epsilon_t = \epsilon_0(1 - t/T)$
- **Exponential:** $\epsilon_t = \epsilon_0 \cdot \beta^t$
- **Step:** Drop at fixed milestones

In finance:

- Early: explore diverse strategies
- Late: exploit best-found policy

Exponential decay ($\beta = 0.995$) is the most common default; linear is simpler to tune

Can an Algorithm Learn to Trade?

Formulation:

- **State**: Price history, portfolio, technical indicators
- **Action**: Buy, sell, hold (+ position size)
- **Reward**: Profit/loss, risk-adjusted return

Challenges:

- Non-stationary environment
- High noise, low signal-to-noise ratio
- Transaction costs
- Partial observability

RL for trading is active research area; not solved problem

How Do You Design the Right Reward Signal?

Reward with transaction costs:

$$r_t = R_t^{\text{portfolio}} - c \cdot |\Delta w_t|$$

where $R_t^{\text{portfolio}}$ = portfolio return, c = transaction cost rate, Δw_t = position change

Common State Features:

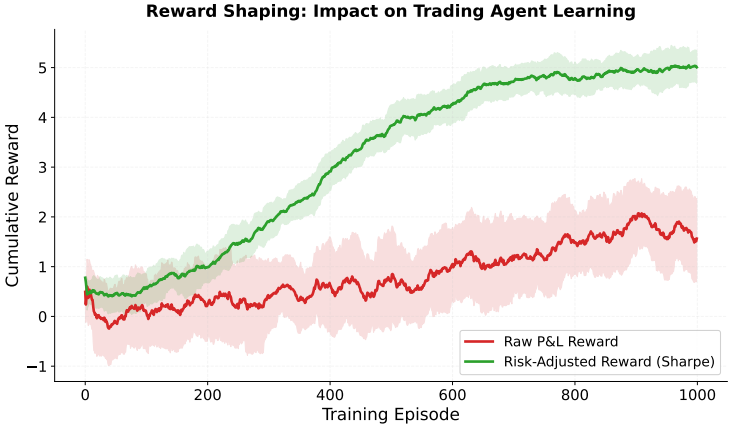
- Price returns (1-day, 5-day, 20-day)
- Technical indicators: RSI, MACD, Bollinger width
- Current position and unrealized P&L

Alternative Rewards:

- Sharpe ratio: $r_t = \frac{\bar{R}_t}{\sigma_{R_t}}$ (risk-adjusted, but non-stationary)
- Log return: $r_t = \log(1 + R_t)$ (additive over time)

Reward design is THE most critical decision in RL for trading

How Much Does Reward Design Affect Learning?



- Same trading environment, two different reward functions
- Raw P&L reward (red): high variance, slow convergence
- Risk-adjusted Sharpe reward (green): smoother, faster learning

Always compare at least 2–3 reward functions before committing to a design

How Do You Know Your RL Strategy Works?

Critical Challenge: RL agents overfit to historical data

Walk-Forward Validation:

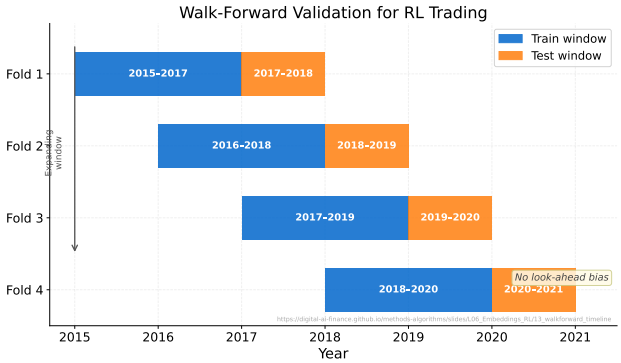
1. Train on period $[t_0, t_1]$, test on $[t_1, t_2]$
2. Roll forward: train on $[t_1, t_2]$, test on $[t_2, t_3]$
3. Report average out-of-sample performance

Honest Evaluation:

- Compare to buy-and-hold benchmark (most RL strategies fail to beat after costs)
- Include realistic transaction costs (0.1–0.5% per trade)
- Test across multiple market regimes (bull, bear, sideways)

If your RL agent beats buy-and-hold after costs, you likely have a bug — verify carefully

How Do You Validate and Deploy a Trading Policy?



Walk-Forward Validation:

- Train on $[t_0, t_1]$, test on $[t_1, t_2]$
- Roll forward and repeat
- Report average OOS performance

Deployment:

- Compare to buy-and-hold benchmark
- Include 0.1–0.5% transaction costs
- Test across bull, bear, sideways regimes

Walk-forward: the gold standard for time-series strategies. If your RL agent beats buy-and-hold after costs, verify carefully.

What If the State Space Is Too Large for a Table?

Problem: Tabular Q-learning stores one value per (s, a) pair

- Trading states (prices, indicators, position) are **continuous** — infinite combinations
- A table with 10^6 states $\times$ 3 actions = 3 million entries (most never visited)

Solution: Deep Q-Networks (DQN)

- Neural network approximates $Q(s, a; \theta) \approx Q^*(s, a)$
- Generalizes across similar states (nearby prices $\rightarrow$ similar Q-values)

Loss Function:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

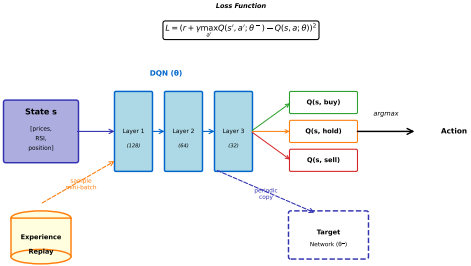
where θ^- are frozen target-network weights (see next slide).

DQN: Mnih et al. (2015) achieved human-level play on 29/49 Atari games; loss is mean squared TD error

How Does DQN Stay Stable?

Two Key Innovations:

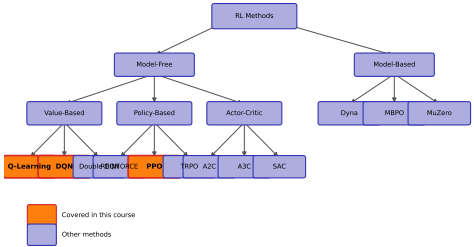
1. **Experience Replay**
   - Store transitions (s, a, r, s') in a buffer; sample random mini-batches
   - Breaks temporal correlation $\rightarrow$ approximately i.i.d. training data
2. **Target Network θ^-**
   - Separate, slowly-updated copy of the network for computing targets
   - Prevents moving target problem: targets don't shift every gradient step



Deep Q-Network Architecture for Trading

Experience replay + target network: the two innovations that made deep RL stable

Where Does Each RL Method Fit?



RL Method Taxonomy

- **Value-based:** Learn Q-values, pick best action (Q-learning, DQN) — *this lecture*
- **Policy-based:** Learn actions directly via gradients (REINFORCE, PPO)
- **Actor-Critic:** Actor picks actions, Critic evaluates — best of both worlds
- **PPO:** Clips policy updates to prevent catastrophic strategy changes

For the Math

See appendix A9 for policy gradient. See L06_rl_full for full PPO derivation.

Orange = covered in this course. Actor-Critic combines value estimation with direct policy learning.

How Do You Practice and Choose the Right Method?

Hands-on Exercises (see Colab notebook):

1. Explore word embeddings with Word2Vec — find similar words and visualize clusters
2. Implement Q-learning on a gridworld and analyze the learned policy
3. Apply RL to a simple trading environment

Decision Rules:

- Text $\rightarrow$ embeddings; sequential decisions $\rightarrow$ RL
- Evaluate embeddings via downstream task F1, not just analogy accuracy
- RL: always compare against buy-and-hold after realistic transaction costs

Both transform complex inputs into learnable representations. Notebook: L06 folder on GitHub.

Key Takeaways

Embeddings:

- Dense vector representations of text/categories
- Capture semantic similarity
- Use pre-trained (Word2Vec, GloVe, BERT)

Reinforcement Learning:

- Agent learns from environment interaction
- Q-learning: value-based, tabular or deep (DQN)
- Applications: trading, portfolio optimization

Key Takeaway: Different tools for different problems

Course complete! Apply these methods in your capstone project

Closing Thought

"After six lectures of methods and algorithms, we've learned the most important lesson:

pour the data into the right pile of linear algebra, and the answers will come out the other side.

The hard part is knowing which pile.'"

— Adapted from XKCD #1838

Callback to XKCD #1838 by Randall Munroe (CC BY-NC 2.5). Course complete!

References

Embeddings:

- Mikolov et al. (2013). Distributed representations of words and phrases. *NeurIPS*, 3111–3119.
- Pennington et al. (2014). GloVe: Global vectors for word representation. *EMNLP*, 1532–1543.
- Devlin et al. (2019). BERT: Pre-training of deep bidirectional transformers. *NAACL*, 4171–4186.
- Araci (2019). FinBERT: Financial sentiment analysis with pre-trained language models. arXiv:1908.10063.

Reinforcement Learning:

- Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Mnih et al. (2015). Human-level control through deep RL. *Nature*, 518, 529–533.
- Watkins & Dayan (1992). Q-learning. *Machine Learning*, 8(3-4), 279–292.
- Schulman et al. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.
- Levine et al. (2020). Offline RL: Tutorial, review, and perspectives. arXiv:2005.01643.
- Liu et al. (2021). FinRL: Deep RL framework for automated trading. *SSRN*.

Sutton & Barto: the definitive RL textbook (free at incompleteideas.net)

Appendix

Advanced Topics and Proofs

Supplementary material for self-study and reference

Appendix slides are not covered in lecture — provided for advanced students and exam preparation.

Skip-Gram Objective Derivation

Maximum Likelihood Objective:
Given corpus of word-context pairs (w_t, w_c), maximize:

$$\mathcal{L} = \sum_{(w_t, w_c) \in D} \log P(w_c | w_t)$$

With softmax parameterization:

$$\log P(w_c | w_t) = v'^{\top}_{w_c} v_{w_t} - \log \sum_{w \in V} \exp(v'^{\top}_{w} v_{w_t})$$

Simplification: The log-sum-exp term is the log-partition function. Maximizing $\mathcal{L}$ is equivalent to minimizing cross-entropy between the model distribution and the empirical context distribution.

Connection to cross-entropy:

$$H(p_{\text{empirical}}, p_{\text{model}}) = -\sum_{w_c} \hat{p}(w_c | w_t) \log p_\theta(w_c | w_t)$$

Skip-Gram is a discriminative model: it models $P(\text{context}|\text{target})$ directly, not a generative process

Negative Sampling Theory

Origin: Noise Contrastive Estimation (NCE)

- NCE (Gutmann & Hyvärinen, 2012): estimate unnormalized models by contrasting data with noise
- Negative sampling is a simplified variant of NCE

Why the 3/4 Power?

- Noise distribution: $P_n(w) \propto f(w)^{3/4}$ where $f(w)$ is unigram frequency
- Exponent < 1 upweights rare words relative to frequency
- Empirically chosen by Mikolov et al. (2013) — not theoretically derived

Implicit Matrix Factorization (Levy & Goldberg, 2014):
SGNS implicitly factorizes a shifted PMI matrix:

$$v_w \cdot v'_c \approx \mathrm{PMI}(w, c) - \log k$$

where $\mathrm{PMI}(w, c) = \log \frac{P(w,c)}{P(w)P(c)}$ and k = number of negatives

Negative sampling implicitly factorizes a shifted PMI matrix

Bellman Equation Convergence

Robbins-Monro Conditions for step sizes α_t:

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t^2 < \infty$$

Contraction Mapping Argument:

- The Bellman optimality operator T is a γ-contraction in ℓ_∞-norm
- $\|TQ_1 - TQ_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty$
- By Banach fixed-point theorem, T has unique fixed point Q^*
- Q-learning converges to Q^* when Robbins-Monro conditions hold and all (s, a) pairs visited infinitely often

Fixed α Issue:

- Constant α violates $\sum \alpha_t^2 < \infty$ — Q-values oscillate around Q^*
- In practice: fixed α works well in non-stationary environments (tracks changes)

Watkins & Dayan (1992): formal convergence proof requires decaying step sizes and full exploration

DQN Architecture Details

Experience Replay Buffer:

- Store transitions $(s, a, r, s', \text{done})$ in buffer of size N (e.g., 10^6)
- Sample uniform random mini-batches for training
- Breaks temporal correlation $\rightarrow$ approximately i.i.d. data

Target Network Updates:

- **Hard update**: Copy $\theta^- \leftarrow \theta$ every C steps (Mnih et al., 2015)
- **Soft update**: $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ with $\tau \ll 1$ (Polyak averaging)

Extensions:

- **Double DQN** (van Hasselt et al., 2016): Decouple action selection from evaluation to reduce overestimation bias
- **Dueling DQN** (Wang et al., 2016): Separate value $V(s)$ and advantage $A(s, a)$ streams:
  $Q(s, a) = V(s) + A(s, a) - \bar{A}(s)$

Mnih et al. (2015): DQN achieved human-level play on 29/49 Atari games

Word Embedding Bias and Fairness

Bias in Word Embeddings (Bolukbasi et al., 2016):

- Gender: he:doctor :: she:nurse (reflects training corpus stereotypes)
- Race, religion, and other protected attributes similarly affected

Debiasing Techniques:

- **Post-hoc projection**: Remove gender direction from embedding space
- **Counterfactual data augmentation**: Balance training examples
- **Adversarial debiasing**: Train to be invariant to protected attributes

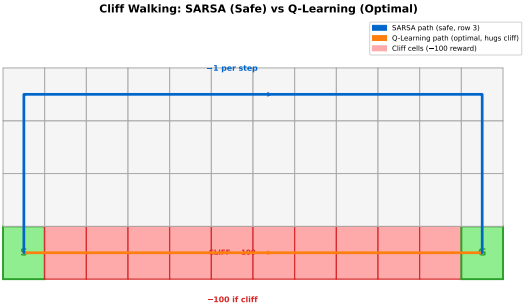
Finance Implications:

- Biased embeddings in **credit scoring** can violate fair lending laws
- **Hiring tools** using biased embeddings risk discrimination claims
- **EU AI Act**: High-risk AI systems (credit, hiring) require bias auditing

---

Embedding bias is a compliance risk in regulated financial services

SARSA vs Q-Learning



SARSA (on-policy):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big]$$

Uses the **actual next action** a'.

Q-Learning (off-policy):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

Uses the **greedy maximum**.

SARSA learns the safe path (accounts for exploration risk); Q-learning learns the optimal but risky path along the cliff edge.

Statistical Inference for Embeddings & RL

Embedding Uncertainty:

- Bootstrap cosine similarity: resample corpus, retrain, compute CI
- Permutation test: shuffle word-context pairs, check if similarity is significant

RL Uncertainty:

- Q-value confidence: run N independent training runs, report mean $\pm$ std
- Off-policy evaluation: importance sampling to estimate policy value from logged data

$$\hat{V}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \prod_{t=0}^{T} \frac{\pi(a_t|s_t)}{\beta(a_t|s_t)} \cdot G_i$$

Always report uncertainty — a single training run is not evidence of a good policy

Policy Gradient Methods

Policy Gradient Theorem (Sutton et al., 2000):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \cdot A^{\pi_\theta}(s,a)\right]$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the **advantage function**

- **REINFORCE**: Uses episode returns G_t as A; high variance
- **Actor-Critic**: Actor (policy π_θ) + Critic (learns V^ϕ); lower variance
- **PPO**: Clips policy ratio to prevent large updates; widely used

Policy gradient handles continuous actions; advantage reduces variance vs raw returns

Appendix References and Further Reading

- Levy, O. & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. *NeurIPS*, 2177–2185.
- Bolukbasi, T. et al. (2016). Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. *NeurIPS*, 4349–4357.
- Watkins, C. & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279–292.
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *AAAI*, 2094–2100.
- Wang, Z., Schaul, T., Hessel, M., et al. (2016). Dueling network architectures for deep reinforcement learning. *ICML*, 1995–2003.
- Sutton, R. & Barto, A. (2018). *Reinforcement Learning: An Introduction*, Chapters 6 and 16. MIT Press.

All appendix references are freely available online